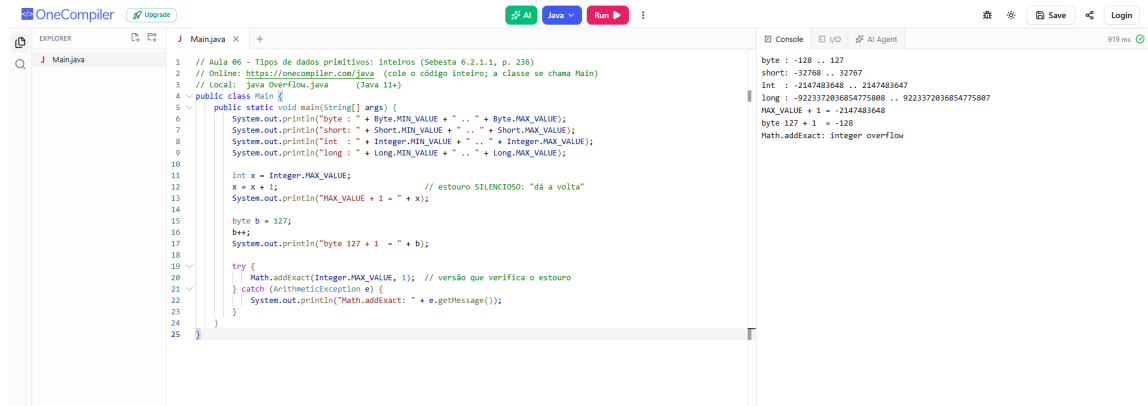


Inteiros



The screenshot shows the OneCompiler Java IDE. The code in `Main.java` demonstrates integer overflow by incrementing `Integer.MAX_VALUE` and then adding 1, which results in a negative value due to overflow. The console output shows the values of `byte`, `short`, `int`, `long`, `MAX_VALUE + 1`, `byte 127 + 1`, and the error message `Math.addExact: integer overflow`.

```
// Aula 06 - Tipos de dados primitivos: Inteiros (Sebesta 6.2.1.1, p. 236)
// Online: https://onecompiler.com/java (cole o código inteiro; a classe se chama Main)
// Local: java Overflow.java (Java 11+)

public class Main {
    public static void main(String[] args) {
        System.out.println("byte : " + Byte.MIN_VALUE + " .. " + Byte.MAX_VALUE);
        System.out.println("short: " + Short.MIN_VALUE + " .. " + Short.MAX_VALUE);
        System.out.println("int : " + Integer.MIN_VALUE + " .. " + Integer.MAX_VALUE);
        System.out.println("long : " + Long.MIN_VALUE + " .. " + Long.MAX_VALUE);

        int x = Integer.MAX_VALUE;
        x = x + 1; // estouro SILENCIOSO: "dá a volta"
        System.out.println("MAX_VALUE + 1 = " + x);

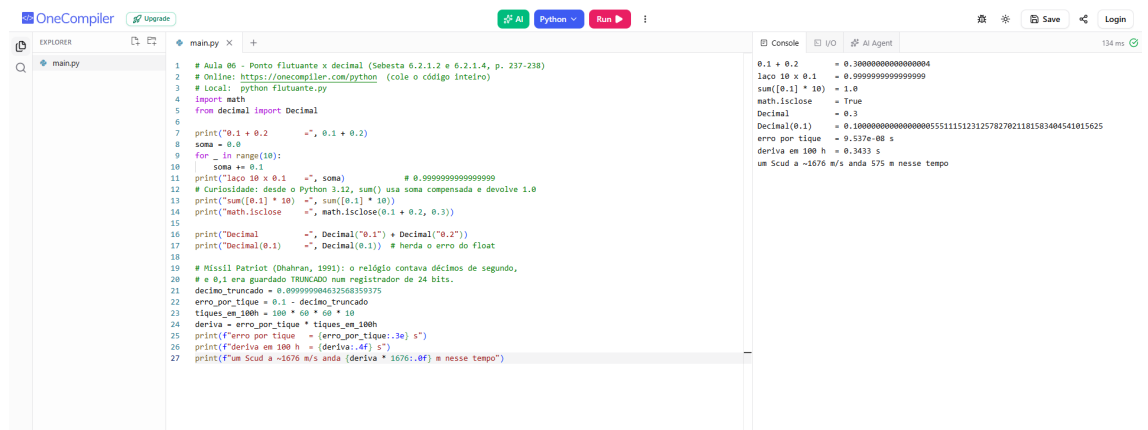
        byte b = 127;
        b++;
        System.out.println("byte 127 + 1 = " + b);

        try {
            Math.addExact(Integer.MAX_VALUE, 1); // versão que verifica o estouro
        } catch (ArithmeticException e) {
            System.out.println("Math.addExact: " + e.getMessage());
        }
    }
}
```

Console Output:

```
byte : -128 .. 127
short: -32768 .. 32767
int : -2147483648 .. 2147483647
long : -9223372036854775808 .. 9223372036854775807
MAX_VALUE + 1 = -2147483648
byte 127 + 1 = -128
Math.addExact: integer overflow
```

Ponto Flutuante



The screenshot shows the OneCompiler Python IDE. The code in `main.py` demonstrates floating-point precision issues by comparing `0.1 + 0.2` with `0.3` and showing the result of `math.isclose`. The console output shows the results of these calculations and the error message `Math.addExact: integer overflow`.

```
# Aula 06 - Ponto flutuante x decimal (Sebesta 6.2.1.2 e 6.2.1.4, p. 237-238)
# Online: https://onecompiler.com/python (cole o código inteiro)
# Local: python flutuante.py

import math
from decimal import Decimal

print("0.1 + 0.2", "=", 0.1 + 0.2)
soma = 0.0
for _ in range(10):
    soma += 0.1
print("10x 0.1", "=", soma) # 0.9999999999999999

# Curiosidade: desde o Python 3.12, sum() usa soma compensada e devolve 1.0
print("sum([0.1] * 10)", "=", sum([0.1] * 10))
print("math.isclose", "=", math.isclose(0.1 + 0.2, 0.3))

print("Decimal", "=", Decimal("0.1") + Decimal("0.2"))
print("Decimal(0.1)", "=", Decimal(0.1)) # herda o erro do float

# Missil Patriot (Dahran, 1991): o relógio contava décimos de segundo,
# e 0.1 era guardado TRUNCADO num registrador de 24 bits.
decimo_truncado = 0.09999999999999999
erro_por_tique = 0.1 - decimo_truncado
tiques_em_100h = 100 * 60 * 60 * 10
deriva = erro_por_tique * tiques_em_100h
print(f"erro por tique = {erro_por_tique:.3e} s")
print(f"deriva em 100 h = {deriva:.4f} s")
print(f"Um Scud a ~1676 m/s anda {deriva * 1676:.0f} m nesse tempo")
```

Console Output:

```
0.1 + 0.2 = 0.30000000000000004
10x 0.1 = 0.9999999999999999
sum([0.1] * 10) = 1.0
math.isclose = True
Decimal = 0.3
Decimal(0.1) = 0.10000000000000000551151231257827021181583404541015625
erro por tique = 9.537e-08 s
deriva em 100 h = 0.3433 s
Um Scud a ~1676 m/s anda 575 m nesse tempo
```

Cadeias



The screenshot shows the OneCompiler JavaScript IDE. The code in `index.js` demonstrates string manipulation by logging the length of a string, the length of its characters, and the length of its code points. The console output shows the results of these calculations and the error message `Math.addExact: integer overflow`.

```
// Aula 06 - Cadeias de caracteres (Sebesta 6.3, p. 240-244)
// Online: https://onecompiler.com/nodejs (cole o código inteiro)
// Local: node cadeias.js

const s = "ação";
console.log(s.length); // 4 (unidades UTF-16)
console.log(s[0].length); // 2
console.log(s[1].length); // 1 (o iterador percorre code points)
console.log(new TextEncoder().encode(s).length); // 6 bytes em UTF-8
console.log(s[1], s.at(-1)); // ç o

s[0] = "A"; // ignorado (em modo estrito: TypeError: string é imutável)
console.log(s); // ação
console.log(s.toUpperCase()); // AÇÃO (nova string)
```

Console Output:

```
4
2
1
6
ç o
AÇÃO
```

Enumerações

OneCompiler

Logado

AI C Run

Save Login

C Main.c

```
1 /* Aula 06 - Tipos enumeração (Sebesta 6.4, p. 245-247)
2 * Online: https://onecompiler.com/c (cole o código inteiro)
3 * Local: gcc enum.c -o enum && ./enum (ou https://godbolt.org)
4 */
5 #include <stdio.h>
6
7 enum Cor { VERMELHO, VERDE, AZUL };
8 enum Fruta { MACA, BANANA, UVA };
9
10 int main(void) {
11     enum Cor c = VERDE;
12     printf("VERDE = %d\n", c); /* 1: em C, enum é só um int */
13
14     int soma = AZUL + UVA; /* aritmética entre "tipos" diferentes */
15     printf("AZUL + UVA = %d\n", soma); /* 4 */
16
17     c = BANANA; /* compila: uma Fruta guardada numa Cor */
18     c = 42; /* compila: valor que nem existe no enum */
19     printf("c = %d\n", c);
20     return 0;
21 }
```

Console

I/O

AI Agent

168 ms

```
VERDE = 1
AZUL + UVA = 4
c = 42
```

Matrizes

OneCompiler

Logado

AI Java Run

Save Login

J Main.java

```
1 // Aula 06 - Matrizes e verificação de índices (Sebesta 6.5.2-6.5.3, p. 249-251)
2 // Online: https://onecompiler.com/java (cole o código inteiro; a classe se chama Main)
3 // Local: java Limites.java
4 import java.util.ArrayList;
5 import java.util.List;
6
7 public class Main {
8     public static void main(String[] args) {
9         int[] v = {10, 20, 30}; /* matriz dinâmica do monte fixa (p. 251)
10         try {
11             for (int i = 0; i <= v.length; i++)
12                 System.out.println("v[" + i + "] = " + v[i]);
13         } catch (ArrayIndexOutOfBoundsException e) {
14             System.out.println("Exceção: " + e.getMessage());
15         }
16
17         // ArrayList: matriz dinâmica do monte (cresce e encolhe)
18         List<Integer> lista = new ArrayList<>(List.of(10, 20, 30));
19         lista.add(40);
20         System.out.println(lista + " tamanho=" + lista.size());
21     }
22 }
```

Console

I/O

AI Agent

1.0s

```
v[0] = 10
v[1] = 20
v[2] = 30
Exceção: Index 3 out of bounds for length 3
[10, 20, 30, 40] tamanho=4
```

Uniãos

OneCompiler

Logado

AI C Run

Save Login

C Main.c

```
1 /* Aula 06 - Uniãos livres (Sebesta 6.10.2, p. 271)
2 * Online: https://onecompiler.com/c (cole o código inteiro)
3 * Local: gcc uniao_livre.c -o uniao_livre && ./uniao_livre
4 */
5 #include <stdio.h>
6
7 union flextype {
8     int intEl;
9     float floatEl;
10 };
11
12 enum Etiqueta { ETQ_INT, ETQ_FLOAT };
13
14 struct Valor {
15     enum Etiqueta etiqueta;
16     union flextype v;
17 };
18
19 int main(void) {
20     union flextype eli;
21     eli.intEl = 27;
22     float x = eli.floatEl; /* nenhuma verificação: lê os bits de 27 como float */
23     printf("x = %g\n", x); /* ~3.78351e-44: lixo */
24
25     eli.floatEl = 1.0f;
26     printf("intEl = %d\n", eli.intEl); /* 1065353216: os bits de 1.0f lidos como int */
27     printf("sizeof = %zu\n", sizeof(union flextype)); /* 4: os campos dividem a memória */
28
29     struct Valor val;
30     val.etiqueta = ETQ_FLOAT;
31     val.v.floatEl = 2.5f;
32     if (val.etiqueta == ETQ_FLOAT) /* a verificação existe, mas depende do programador */
33         printf("float %g\n", val.v.floatEl);
34     return 0;
35 }
```

Console

I/O

AI Agent

188 ms

```
x = 3.78351e-44
intEl = 1065353216
sizeof = 4
float 2.5
```

Ponteiros

OneCompiler

Upgrade

AI

Java

Run

Manjava

```
1 // Aula 06 - Tipos de referência e null (Sebesta 6.11.5-6.11.6, p. 278-280)
2 // Online: https://onecompiler.com/java (cole o código inteiro; a classe se chama Main)
3 // Local: Java Referencias.java
4 import java.util.Optional;
5
6 public class Main {
7     static class Conta {
8         double saldo;
9         Conta(double saldo) { this.saldo = saldo; }
10    }
11
12    static Optional<String> buscarNome(int id) {
13        return id == 1 ? Optional.of("Ana") : Optional.empty();
14    }
15
16    public static void main(String[] args) {
17        Conta a = new Conta(100); // a e b referenciam o MESMO objeto (apelidos)
18        Conta b = a;
19        b.saldo = 0;
20        System.out.println("a.saldo = " + a.saldo); // 0.0
21
22        a = null; // o objeto ainda é alcançável por b
23        b = null; // agora é lixo: o coletor (GC) libera sozinho
24
25        String nome = null;
26        try {
27            System.out.println(nome.length());
28        } catch (NullPointerException e) {
29            System.out.println("NullPointerException: " + e.getMessage());
30        }
31
32        // Optional: a "ausência de valor" vira parte do tipo
33        System.out.println(buscarNome(1).map(String::length).orElse(0)); // 3
34        System.out.println(buscarNome(2).map(String::length).orElse(0)); // 0
35    }
36 }
```

Console

AI Agent

1.1s

```
a.saldo = 0.0
NullPointerException: Cannot invoke "String.length()" because "<local>" is null
3
0
```

Exercicio

OneCompiler

Upgrade

AI

JavaScript

Run

index.js

```
1 // Exercício Aula 06 - Questão 1
2 // ANTES de executar: qual é a saída? O problema é detectado na compilação, na execução ou nunca?
3 // Online: https://onecompiler.com/nodejs (cole o código inteiro)
4 // Local: node q1.js
5 console.log(0.1 * 3 === 0.3);
6 console.log(9987199254748992);
```

Console

AI Agent

161 ms

```
false
9987199254748992
```

Autopesquisa

OneCompiler

Upgrade

AI

Python

Run

main.py

```
1 # Autopesquisa - Matrizes associativas (Sebesta 6.6, p. 259-263)
2 # Online: https://onecompiler.com/python (cole o código inteiro)
3 # Local: python associativas.py
4 idades = {"ana": 20, "bia": 22}
5 idades["caio"] = 19
6 print(idades.get("zeca", "não existe"))
7 for nome, idade in idades.items(): # desde o Python 3.7, mantém a ordem de inserção
8     print(nome, idade)
9 del idades["bia"]
10 print("bia" in idades, idades)
```

Console

AI Agent

139 ms

```
não existe
ana 20
bia 22
caio 19
False ('ana': 20, 'caio': 19)
```